Семинар 4. Основы OpenMP

1 Введение

OpenMP представляет собой библиотеку и модификацию компилятора. OpenMP использует модель общей памяти, к которой имеют доступ все процессоры. При написании программ на OpenMP широко используются директивы препроцессора.

Данная библиотека совместно с MPI формирует «золотой стандарт» в мире высокопроизводительных вычислений: в то время как MPI используется для организации коммуникаций между узлами кластера, отдельные компьютеры в рамках гетерогенной системы используют OpenMP для многопоточных вычислений.

Т.к. OpenMP не только библиотека, но и расширения компилятора, отдельные компиляторы могут не поддерживать эту библиотеку. Если компилятор не поддерживает OpenMP, то все директивы, адресованные этой библиотеки, будут компилятором игнорироваться. Однако вызов функций из состава OpenMP приведет к ошибке компиляции. Поэтому для написания портабельного кода нужно заключать вызовы функций OpenMP в блок условной компиляции:

```
#ifdef _OPENMP
// call to OpenMP functions
#endif
```

2 Создание потоков

Потоки создаются с помощью следующей директивы:

```
#include<omp.h>
...
```

```
#pragma omp parallel num_threads(thread_count)
Structured block
```

Структурный блок представляет собой одну или несколько команд. В случае, если операторов больше одного, их нужно заключить в операторные скобки {}.

Директива `barrier` позволяет осуществить барьерную синхронизацию между всему потоками. Отметим, что директивы `parallel, parallel for` и `for` (о двух последних поговорим позже) содержат в себе неявный барьер в конце структурного блока. Поэтому в листинге 1 нет явного барьера после структурного блока: главный поток будет ждать, пока все остальные потоки завершат свою работу.

Листинг 1: Многопоточный «Hello, world» на OpenMP

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv){
  int thread_count = strtol(argv[1], NULL, 10);

#pragma omp parallel num_threads(thread_count)
{
  int my_rank = omp_get_thread_num();
  printf("Hello from thread %d from %d\n",
  my_rank, thread_count);
}
}
```

Область видимости переменных в `OpenMP` может быть `shared` и `private`. По умолчанию переменная, объявленная до директивы `parallel`, является общей. Любая переменная, объявленная внутри структурного блока, является приватной для потока. Можно потребовать, чтобы области видимости всех переменных задавались явно, для этого нужно использовать директиву `default(none)`. Единственным исключением является переменная редукции (см. далее), статус которой определяется автоматически и не может быть задан директивно.

t	1	2
1	Получить	—
2	Сравнить экспоненты	Получить
3	Сдвинуть запятую	Сравнить экспоненты
4	Сложить мантиссы	Сдвинуть запятую
5	Округлить	Сложить мантиссы
6	Записать	Округлить
7	—	Записать

Таблица 1: Механизм возникновения состояния гонки

3 Критические секции

При изменении общей переменной несколькими потоками возникает состояние гонки, что приводит к искажению результатов. Состояние гонки возникает в том случае, когда второй поток начинает изменять переменную после того, как первый поток начал ее изменять, но до того, как первый поток закончил.

Проиллюстрируем механизм возникновения состояния гонки на примере сложения двух чисел с помощью операции `sum += a`, где `sum` — общая переменная. См. таблицу 1.

Часть кода, одновременное выполнение которой несколькими потоками приводит к возникновению состояния гонки, называется критической секцией. В OpenMP существует несколько способов исключить одновременное выполнение критической секции несколькими потоками. Самый простой способ — использование директивы `#pragma omp critical`.

4 Редукция

Следующий фрагмент кода

```
global_result = 0.0;
#pragma omp parallel num_threads(thread_count)
{
  double my_result = 0.0;
  my_result += f();
#pragma omp critical
  global_result += my_result;
```

```
}
```

является часто используемым паттерном и может быть идиоматически записан в следующем виде:

```
global_result = 0.0;
#pragma omp parallel num_threads(thread_count)\
    reduction(+: global_result)
global_result += f();
```

Данная более компактная запись называется редукцией сложения по переменной `global_result`. Редукция может быть применена не только к сложению, но и вообще к любой бинарной ассоциативной операции: арифметической, логической или побитовой.

Синтаксис директивы редукции: `reduction(op : var_list)`,
где `op` — операция (+, -, *, /, &&, &, |, ||, ^);
`var_list` — список переменных редукции через запятую.

Редукция работает следующим образом. В случае изменения одной из переменных из списка `var_list` создается временная приватная переменная, инициализируемая нейтральным элементом по операции `op` и все дальнейшие действия происходят с приватной переменной. После того, как все потоки завершают работу, в общую переменную под директивой `critical` записываются результаты каждой из приватных переменных, причем к ним применяется операция редукции `op`. Исключение составляет вычитание. В этом случае к частичным результатам все равно применяется сложение.

Проиллюстрируем применение директивы `critical` и редукции к задаче вычисления значения определенного интеграла. См. листинг 2.

Листинг 2: Вычисление определенного интеграла

```
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>

double f(double x){
        return x*x;
}

double local_trap(double a, double b, int n){
```

```
        int my_rank = omp_get_thread_num();
        int thread_count = omp_get_num_threads();
        double h = (b-a)/n;
        int local_n = n / thread_count;
        double local_a = a + local_n*h*my_rank;
        double local_b = local_a + local_n*h;
        double my_result = 0.5*(f(local_a)+f(local_b));
        for(int i = 1; i < local_n; ++i){
                double x = local_a + i*h;
                my_result += f(x);
        }
        my_result *= h;
        return my_result;
}

int main(int argc, char** argv){

        int thread_count = strtol(argv[1], NULL, 10);

        double global_res = 0.0;
#pragma omp parallel num_threads(thread_count) reduction(+: global_res)
        global_res += local_trap(0,3,1000);
        printf("%.15lf\n", global_res);

}
```

5 Замеры времени выполнения алгоритма

Перед началом замера нужно поставить барьер. Замеряем время только самого параллельного алгоритма, ввод и вывод мы не параллелим, следовательно, не включаем во время работы.

```
#pragma omp barrier
double start = omp_get_wtime();
// algo
double end = omp_get_wtime();
```